A Hybrid Approach for Detecting Fraudulent Bitcoin Transactions Based on Graph-Structured Data

Arne Mallon
Department of Informatics
University of Hamburg
Hamburg, HH 22527
arne.mallon@studium.uni-hamburg.de

Josef Hermann
Department of Informatics
University of Hamburg
Hamburg, HH 22527
josef.hermann@studium.uni-hamburg.de

Lukas Owen
Department of Informatics
University of Hamburg
Hamburg, HH 22527
lukas.owen@studium.uni-hamburg.de

October 1, 2025

Contents

1 Introduction **3**
1.1 Problem . . . 3
1.2 Objective . . . 3

2 State of Research **3**

3 Business Understanding **4**

4 Data Understanding and Preparation **5**
4.1 Overview . . . 5
4.2 Original Dataset . . . 5
4.3 Graph Construction and Subgraph Extraction . . . 5
4.4 Feature Preparation . . . 6
4.4.1 Preprocessing . . . 6
4.4.2 Feature Extraction . . . 6
4.4.3 Feature Selection . . . 7
4.5 Final Dataset . . . 7

5 Modeling **8**

6 Evaluation **9**

7 Deployment **9**

8 Discussion **10**

9 Conclusion **10**

A Calculations for Feature Selection **12**
A.1 Chi-squared test . . . 12
A.2 F-test . . . 12
A.3 Pearson Correlation Coefficient . . . 12
A.4 Mutual Information . . . 13

1 Introduction

1.1 Problem

Bitcoin is a peer-to-peer network that enables decentralized and secure transactions. Since this network is decentralized and therefore not directly connected to authorities, it constitutes an attractive platform for criminals to complete transactions. Although it is often assumed that these transactions are completely anonymous, various details of every transaction ever made can be traced without much effort on the World Wide Web.

In order to combat the use of Bitcoin for the financial infrastructure of criminal organizations, crypto exchanges are obliged to take measures to prevent such activities on their platforms. In addition to a know-your-customer procedure, they also check the origin of funds, i.e., whether these Bitcoins have already been used for criminal purposes or stolen (European Commission, 2024a; 2024b). There are already various approaches in existence to solve this problem, but they are based on the fact that certain addresses have already been classified and then simply tracing the transaction flow. Often, this procedure is executed manually, for example, by law enforcement agencies. Existing approaches, therefore, require at least one address from a cluster of illegal Bitcoin addresses from the outset, and the process remains time-consuming and inefficient.

Bitcoin transactions can have multiple inputs and outputs, whereby the total balance of all input addresses should be distributed entirely to the output addresses; otherwise, the difference would be retained as a transaction fee (Nakamoto, 2008). Over the years, techniques have been developed to make it as difficult as possible to trace funds. Tumblers are often used for this purpose. Tumblers are a construct in which a large number of addresses, some of which are involved in criminal activities, are used in a transaction to conceal the flow of money. The many outputs, therefore, no longer allow a specific input to be identified. Transactions are often routed through tumblers several times, making it almost impossible to trace the flow of Bitcoin. So even if the criminal nature of funds is known, it is almost impossible to trace them back to their origin. (Andersson, 2024)

Centralized exchanges are legally required to block wallets if there is suspicion that Bitcoin from criminal activities is being converted into cash. If they do not adhere to this in an effective manner, criminals are able to continue to expand their financial infrastructure, damaging the reputation of decentralized finance.

1.2 Objective

Among other things, our solution should enable centralized exchanges and law enforcement agencies to classify addresses more quickly, with less manual work and without the need for the premise of a known address from a criminal cluster. No need to try to track addresses through multiple tumblers should remain; instead, these patterns can be recognized in advance. The analysis should solely require the wallet address as input for the classification in order to guarantee maximum operational simplicity for our platform.

We want to provide an approach that can be further developed and perfected to curb criminal activity on the Bitcoin blockchain.

2 State of Research

The initial approach and the data set we use are based fundamentally on the article "BABD: A Bitcoin Address Behavior Dataset for Pattern Analysis" by Xiang et al. (2022). This article describes how the authors proceeded to create a transaction graph from Bitcoin transactions and extract various features from it. These features form the basis for the dataset published on Kaggle. The authors focus primarily on structural and statistical parameters such as PageRank or in- / out-degree (Saeidimanesh, 2024, pp. 10–13). However, this leads to a significant loss of information, as each address is considered in isolation, thus largely ignoring complex relationship information between addresses and insufficiently taking into account the network topology (Xiang et al., 2022).

Therefore, our goal is to generate node embeddings for the graph. A node embedding is a vector that reduces high-dimensional information about a node's neighborhood and the structure of a graph to a lower dimension. This captures both local and global patterns of the network and allows the complete graph structure to be integrated into the modeling. The publication "Demystifying Bitcoin Address Behavior via Graph Neural Networks" demonstrates that methods based on such embeddings can significantly outperform classical approaches that only consider structure-related metrics (Huang et al., 2022).

We investigated two methods for generating node embeddings: first, the node2vec algorithm (Grover et al., 2016). This involves performing random walks from each node to capture structural information in the form of vectors. The algorithm is semi-supervised, which means that only a small amount of initial training is required, and it can be flexibly adjusted using parameters. Several node2vec frameworks were proposed in the literature (Hu et al., 2019; Çağlayan et al., 2022; Yuan et al., 2020; Liu et al., 2023) and have shown the significant disadvantage that they work transductively. This implies that the model must be retrained on the entire graph for each new address or transaction, which would be very time-consuming in our case.

We investigated the GraphSAGE algorithm (Hamilton et al., 2017) as a second method. GraphSAGE is a variant of Graph Convolutional Networks (GCN). It works inductively and captures graph structures. In this way, new node embeddings can be efficiently calculated for previously unseen nodes or subgraphs. Another advantage is that GraphSAGE takes into account not only structural relationships but also the distribution of node features, which is particularly relevant for our dataset. In contrast, the training effort is significantly higher because GraphSAGE requires its own machine learning model.

There have been publications regarding the inclusion of graph data in the past. However, they did not leverage the information contained in the graph to its full potential.

One paper titled "Demystifying Fraudulent Transactions and Illicit Nodes in the Bitcoin Network for Financial Forensics" (Elmougy et al., 2023) used the graph for feature extraction and visualization purposes and trained the machine learning models on tabular data containing the extracted features. Yet, they have ignored the information contained in the graph structure itself and therefore lost out on significant information.

Another publication with the title "Anti-Money Laundering in Bitcoin: Experimenting with Graph Convolutional Networks for Financial Forensics" (Weber et al., 2019) proposed the use of GCN to capture relational information. The authors drew attention to the arising challenges that come with the use of this methodology. They decided to solve the problem of the heterogeneity in the number of neighbors of the nodes by "constructing statistical aggregates [...] of the local features of a neighbor transaction" (Weber et al., 2019, p. 3). This way, the substantial disadvantage of their procedure was addressed to prevent information loss. In the end, they proposed the question: "Is it possible to combine a Random Forest with a graph neural network?" (Weber et al., 2019, p. 6). It is our intention to take up this question and find an approach that allows for higher accuracy by combining structural features from a GCN with random forests.

The popular Elliptic Bitcoin dataset proposed by Weber et al. (2019), despite being one of the most comprehensive crypto transaction datasets available, has foundational limitations that are also passed on to the studies that use it (Asiri and Somasundaram, 2025; Elmougy and Liu, 2023). These limitations are two-fold. Firstly, approaches using this dataset often fail to make use of the rich topological information of the transactions, which could be preserved by complementary graph data such as embeddings (Asiri and Somasundaram, 2025; Elmougy and Liu, 2023). Secondly, limited by the nature of the Elliptic dataset that only maps transactions as licit and illicit (Elliptic, 2019), these studies are solely able to give a binary classification, lacking deeper insight into the Bitcoin network criminality that would be priceless to stakeholders.

3 Business Understanding

The two main stakeholders of our project are law enforcement agencies and regulated cryptocurrency exchanges. The former fight financial crime such as money laundering and specify regulations that financial institutions, particularly these exchanges, have to comply with.

Entities, including banks and cryptocurrency exchanges, have to apply measures to prevent money laundering and terrorist financing as determined by the Anti Money Laundering Framework (AML) in the European Union and the Financial Crimes Enforcement Network (FinCEN) for the United States. Additional regulations apply to cryptocurrency exchanges, which aim to mitigate key risks in connection with the crypto asset market, such as reducing the risk of fraud, market manipulation and money laundering as part of the Markets in Crypto-assets Regulation (MiCA) as well as a new Regulation on the Traceability of Transfers of Funds (TFR) for the European Union as of December 2024. These Frameworks ensure the traceability of crypto-assets transfers and implement a Know Your Customer Policy that demands tracing the source of funds and monitoring account activity. Crypto asset service providers like exchanges have to comply with these frameworks to get regulated and obtain approval in their respective regions. Most of these frameworks are heavily reliant on the Financial Action Task Force (FATF) and vary from country to country. Yet, finding and mitigating acts of money laundering remains a key point (European Commission, 2024a; 2024b).

Since the business model of these exchanges is not centered on preventing criminal activities, they want to use as few resources and manual labor as possible to fulfill the requirements. Therefore, achieving fast results at low cost with minimal manual labor is crucial to their operations. Law enforcement agencies, on the other hand, prefer an emphasis on accuracy of their investigation and less on minimizing costs. For them, we will have to provide an application that delivers reliable results. Herefrom we can derive the criteria of success for this project. In addition to the previously mentioned criteria, the application should be scalable, allowing users to review multiple addresses at the same time.

Both actors will mainly use the application to investigate suspected cases. We should also provide the possibility to track a list of addresses to regularly check their activities. When a suspicious address is found, we should also check its neighbors to likely find more addresses involved in criminal activity. Analyzed addresses will be categorized into 13 different categories as specified in the BABD-13 paper. These categories are blackmail, cybersecurity service, darknet market, centralized exchange, P2P financial infrastructure service, P2P financial service, gambling, government criminal blacklist, money laundering, Ponzi scheme, mining pool, tumbler, and individual wallet (Xiang et al., 2022). These categories not only differ in criminal extent but also in the methods and related difficulty and effort. For example, classifying centralized exchanges is fairly easy. We analyzed the crypto exchange wallets from the BABD-13 dataset (Xiang et al., 2022) and found that, next to high volume and transaction count, they show an average input-to-output ratio of 1.004. This demonstrates that wallets from centralized exchanges share the very common and quite unique property that the total amount of Bitcoin that was sent to the wallet is very close to the amount sent from the wallet. Gambling wallets, on the other hand, usually possess a higher volume of low-value incoming transactions and a lower volume of high-value outgoing transactions, which can be attributed to gamblers betting smaller amounts of money and gaining mostly bigger payouts from time to time (Conlon et al., 2020). As described in the introduction, tumblers obscure the flow of funds by combining a large number of both criminally used and normal addresses in the input and dis-

tributing them to a large number of outputs for a single transaction. This suggests a wallet with a high number of transactions as well as distinct addresses may indicate the use of a tumbler (Andersson et al., 2024).

As for the business model of our project, we distinguish between different monetization strategies depending on the type of user. While law enforcements pay per address analyzed, regulated exchanges pay a seat-based subscription fee. This approach allows us to monetize the respective user behaviors in an optimal manner. Law Enforcement agencies are willing to pay more to find suspicious addresses, reflecting their intention to investigate accurately, whereas exchanges do not directly benefit from finding criminal activity and thus mainly profit from predictable, low costs.

A risk assessment revealed several potential risks for this project. First, we are dependent on the APIs we use. We have to consider their availability and rate limit; thus, we should have a backup strategy in case one API fails, to fetch the necessary information fast enough. Second, we derive the best results if the analyzed address is included in our graph, with which we generated the embeddings. If it is not, adding the new address and retraining on the graph is possible, but very time-consuming and computationally intensive. Third, we have to make sure that underrepresented categories are not getting overlooked, which would lead to biased results. Lastly, we have to ensure we mitigate false positive results, which are more severe than false negatives, since it is worse to freeze an innocent account than to miss a single address involved in crime, which would result in future trust issues and conflicts with the business model of the exchanges.

Even though this project is focused on the Bitcoin network, the proposed strategies and technologies can be transferred to different blockchains that are also subject to money laundering, like Ethereum / ERC-20 tokens. This adaptation could provide a promising opportunity for the application to integrate cross-chain analysis that includes money flows that switch between different cryptocurrencies to ensure its remaining relevance for its stakeholders. However, some networks like Monero and Zcash hide their transaction data, making it impossible to track transactions and discover money laundering, for which these technologies can not be applied (Scharnowski, 2024).

4 Data Understanding and Preparation

4.1 Overview

As depicted in Figure 1, we followed a multi-step procedure to construct the final data set. The starting point of our data preparation was the BABD-13 dataset (LEMONX, 2022), whose well-documented generation we built upon by reconstructing the Bitcoin address-transaction graph for the relevant blocks. We then restricted this graph to the intersection of the in BABD-13 labeled addresses and all the addresses in the reconstructed graph. Taking these intersection nodes as a seed, we constructed the 2-hop graph around them on which we trained GraphSAGE to obtain structural embeddings. Finally, the structural embeddings

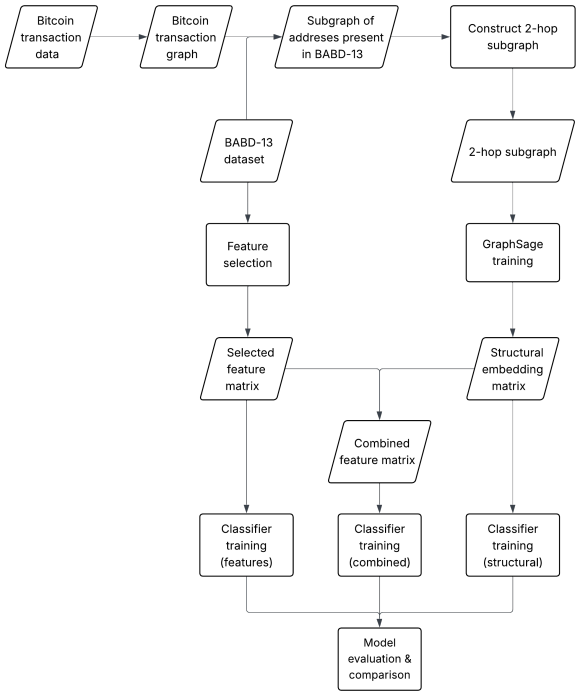

Figure 1: Overall methodological workflow

were combined with the selected features from the BABD-13 dataset for the intersecting addresses to constitute our final feature matrix. The feature matrices created during this process, namely solely the embeddings as well as solely the from BABD-13 selected features, were retained as well and used for our machine learning models as a reference.

4.2 Original Dataset

The original dataset "BABD-13" contains 13 types of Bitcoin addresses, 5 categories of indicators with 148 features, and 544,462 labeled data (LEMONX, 2022). It is a highly imbalanced dataset with the most present label claiming 55.1% of the data (label 3) , whereas the least occurring only claims 0.002755% (label 9). Despite the mention "Partial Bitcoin ledger data from height 600,000 to 605,999 are available here" on the dataset's kaggle website as well as the Bitcoin blocks 600,000 to 605,999 being used in the script for the graph construction on their official github repository (Y-Xiang-hub, 2022) that documents the construction of BABD-13, the used blocks for the creation of the dataset are from the height 585,000 to 685,000 as described in the BABD paper.

4.3 Graph Construction and Subgraph Extraction

We reconstructed the complete address-transaction graph for the block range 600,000 to 605,999 as documented in the BABD-13 GitHub repository. The full graph contains 31,773,213 vertices in total, of which 18,679,985 are ad-

dress nodes, 59,929,628 edges, and 13,093,228 transaction nodes. Due to the unclear and misleading documentation of the use of Bitcoin blocks for the generation of the BABD-13 dataset on the Kaggle website, as well as the GitHub repository, only 94,966 of the 544,462 BABD-13 addresses are contained in our reconstructed approximately 18 million graph address node transaction graph. Using these labeled address nodes as seeds, we constructed the 2-hop subgraph by expanding within the complete transaction graph, including all the nodes reachable within two hops from the seeds. Due to our limited computational resources, we chose 2 hops as a compromise between computational cost and deep structural information. The resulting graph contains 1,975,895 vertices, of which 1,617,111 are address nodes and 5,634,708 edges.

4.4 Feature Preparation

4.4.1 Preprocessing

In order to extract only the most meaningful information from our data, a structured preprocessing pipeline was applied. This ensured that the dataset used for model training and evaluation was both consistent and of high quality. First, all empty features were removed from the dataset. For the remaining features, missing values were imputed using the median, which was calculated exclusively on the training split in order to avoid data leakage.

Subsequently, features with no variance (constant features) were discarded, as they provide no discriminative value for the models. To make all features comparable, they were scaled by means of standardization, transforming them to have a mean of zero and a variance of one. Similar to the imputation step, the scaling procedure was fitted only on the training data and then applied to all nodes.

Finally, due to the pronounced class imbalance, classes with fewer than 100 labels were removed in order to prevent distortions caused by extremely rare classes and ensure meaningful modeling. This leaves a total of 94776 distinct addresses.

Table 1: Labels Dropped Due to Insufficient Sample Size

Label	Name	Sample Count
4	P2P financial infrastructure service	65
10	mining pool	65
12	individual wallet	56
9	Ponzi scheme	3
7	government criminal blacklist	1

4.4.2 Feature Extraction

One of the important features we utilize is the aforementioned node embeddings. To generate meaningful node embeddings, we employed the GraphSAGE framework. The name stems from the two core processes utilized by this Algorithm: sampling and aggregation. As the name suggests, the first step to creating node embeddings is sampling their neighborhood for features. We employ fixed-size random neighbor sampling. In this approach, a fixed number of neighbors is randomly selected for each node. This ensures constant runtime and memory requirements, independent of the node degree.

Secondly, in order to aggregate the information from the sampled neighbors, we opted for the max-pooling aggregation. While GraphSAGE supports different aggregation schemes such as mean, LSTM-based, and max-pooling, the latter was chosen because it highlights dominant feature expressions of the neighbors and is comparatively robust to noise. This method has been shown to work particularly well for cases like ours where informative node attributes are available. Since GraphSAGE combines both the features of nodes and their neighborhoods, the quality of input features plays a crucial role. In our case, the pre-existing node attributes provided valuable context for the embedding generation.

The dataset was split into training (60%), validation (20%), and test (20%) sets using a stratified split strategy. Our setup is distinct in that it follows an inductive learning paradigm, meaning that the node sets are disjoint with no overlap between training, validation, and test nodes. To enforce this separation, edges between subgraphs were masked. The training subgraph, therefore, consisted only of training nodes and their neighbors (excluding validation and test nodes), while validation and test subgraphs were built separately in the same manner. This ensures that the model never encountered validation or test nodes during training.

Additionally, we utilize a three-layer GraphSAGE encoder. Our architecture allowed the model to aggregate information up to the 3-hop neighborhood of each node. Thereby, a key advantage of GraphSAGE is its inductive capability, meaning it can generalize to previously unseen nodes. The encoder produces 32-dimensional node embeddings. We selected a dimensionality of 32 in order to balance expressiveness and efficiency. On the one hand, embeddings must be sufficiently high-dimensional to capture both structural and attribute-based information of the graph. On the other hand, excessive dimensionality can lead to overfitting, increased computational overhead, and reduced interpretability of the representations.

To further improve embedding quality, we applied Deep Graph Infomax (DGI), a self-supervised approach for representation learning. The underlying concept is to contrast positive and negative examples: positive samples consist of real node features, while negative samples are created by randomly permuting node features, resulting in a corrupted version. The model is trained to distinguish between these two cases, which encourages the embeddings to capture global structural properties of the graph. Training was performed using the Adam optimizer, with early stopping based on the macro-F1 score of a logistic regression probe on the validation set.

After training, embeddings were extracted for all training, validation, and test nodes. For each node list, a subgraph was built up to the required neighborhood depth, and the encoder computed the embeddings accordingly. Finally, all embeddings were stored in both CSV and Parquet format for downstream usage.

4.4.3 Feature Selection

We started the feature selection process for the original BABD-13 features by applying a low variance filter and removing all features with a variance below 0.01. For validation after selection, we split the remaining feature matrix into a training set (80%) and a test set (20%), utilizing a stratified split which retains class distribution. For more details on the scores and their calculations, see Appendix A. With this preparation, we used the absolute values to calculate the chi-squared score, which measures statistical independence between feature and label. Next, we calculated the score of the F-test that measures how discriminatory the features are, i.e., how strongly features differ between classes. Lastly, we assessed the Pearson correlation, which serves as an indicator of the linear correlation between feature and label. Since only the strength of the correlation is relevant to our analysis and not the direction, we calculated the absolute values of these results, which could be both negative and positive.

After computing the three independent metrics, a feature superset was constructed by combining the top 20 features from every score. Subsequently, a correlation matrix was computed, and the weaker feature, according to our previous selection, from each feature pair, where the correlation lies beyond our threshold of 0.8, was eliminated.

To also recognize non-linear relationships between features and labels, we calculated the mutual information, which measures how much information a feature contains about the label. Next, we assessed the random forest feature importance by training a random forest with 100 trees and measuring the importance of each feature for the classification, which also measures non-linear relationships.

The final step of the feature selection process consisted of aggregating the 5 different scores. Because the scores differed significantly in magnitude, they were first normalized in order to ensure comparability. This yielded a final ranking of features from which the top eight were selected as follows: (in descending order of their normalized total score):

Table 2: Top 8 Selected Features and Their Descriptions

Feature	Description
S5	Average shortest path length between nodes
S6	Maximum diameter of the graph
S1-1	Average in-degree
PTIa41-2	Minimum time interval between two consecutive transactions of an address node
CI2a32-2	Maximum ratio of the change in total input token amount of an address node to its transaction time intervals
PTIa21	Ratio of the number of active days to the total lifecycle duration of an address node
PAIa13	Ratio of the total input token amount to the total output token amount of an address node
CI3a12-3	Minimum daily in-degree of an address node during its active days (Xiang et al., 2022)

Figure 2 shows the correlation matrix for the selected features. Evidently, the highest correlation between two features lies with S5 and S6. Next to S1-6, they also exhibit the highest correlation with the target label, suggesting the highest value for classification.

Using these 8 features, a random forest classifier with 100 decision trees was trained, achieving a test accuracy of about 91.07%. This indicates the soundness of our approach, which provides a test score that is far exceeding results obtained from guessing randomly (8.33%) or always guessing the majority class (64.70%). Given their demonstrated predictive value, these features were subsequently used for all further model training.

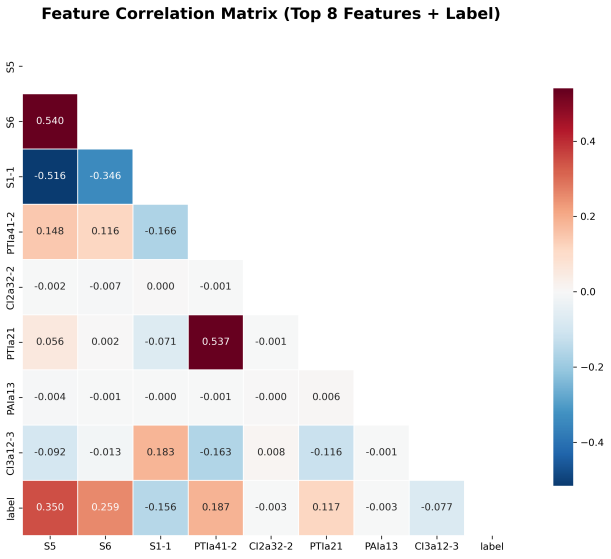

Figure 2: Feature correlation matrix of top 8 features

4.5 Final Dataset

To construct the final dataset, the structural embeddings obtained with GraphSAGE and the selected original features from the BABD-13 dataset were concatenated, considering only the intersection of addresses present in both BABD-13 and our reconstructed transaction graph. This resulted in a combined feature matrix of the size $94{,}776 \times 42$, where the first dimension corresponds to the number of distinct addresses and the second consists of 32 embeddings, 8 original features from BABD-13, the account type, and the label. Before being passed to downstream models, the merged features were once again standardized using a StandardScaler fitted exclusively on the training split in order to avoid leakage and subsequently applied to validation and test splits. Moreover, to further address the remaining class imbalance in the dataset, we considered two approaches, namely random undersampling and class weighting, which were applied depending on the respective model, as detailed in the Modeling section.

5 Modeling

Through the data preparation, we have constructed a comprehensive feature space. The final dataset ensured that both the structural information captured through the embeddings and the attribute-based information from the original dataset were leveraged simultaneously. Before being passed to downstream models, the merged features were once again standardized using a StandardScaler fitted exclusively on the training split in order to avoid leakage and subsequently applied to validation and test splits.

Before training models on the dataset, addressing the inherent class imbalances is an important issue. For the Random Forest classifier, random undersampling was applied to the training split using the RandomUnderSampler implementation from imbalanced-learn. This procedure randomly removes samples from the majority classes until all classes are equally represented, ensuring a uniform class distribution for training. In addition, class weights were applied, computed via scikit-learn's compute_class_weight function. These weights assigned larger importance to minority classes and reduced the influence of majority classes during optimization. As for the Boosting models, only class and sample weights were used, ensuring that rare classes received stronger emphasis without discarding valuable training data. In addition, per-sample weights were computed for both kinds of models so that examples from rare classes counted more during training. This ensured that errors on minority classes had a stronger impact on model optimization, mitigating the risk of the model favoring dominant classes. Undersampling was applied exclusively to the Random Forest, as it helps mitigate bias toward majority classes in bagging-based methods that do not natively handle severe imbalance. In contrast, boosting models such as XGBoost and LightGBM iteratively focus on difficult-to-classify samples. For these methods, discarding data through undersampling would remove valuable training signals and reduce their effectiveness. Therefore, class and sample weighting provided a more suitable approach for boosting models, ensuring balanced learning while preserving the full dataset.

The overall training procedure was structured as follows: The dataset was consistently partitioned into training, validation, and test splits according to the original node sets created during embedding extraction, preserving the inductive learning setup. After feature fusion and scaling, hyperparameter optimization was conducted via cross-validation on the training split, using the macro-F1 score as the optimization criterion. The configuration achieving the highest cross-validation score was selected as the best-performing setup. For the Random Forest, the model was evaluated on the validation split both with and without balancing techniques (undersampling and class weights) to assess their effect on generalization, while for gradient boosting models, only class weights were applied to preserve the full training data. For the final stage, the model was retrained on the combined training and validation data, without balancing methods, and its performance was evaluated on the previously unseen test set.

A Random Forest classifier was trained as a baseline. Here, the hyperparameter search focused on two key factors: the number of decision trees in the ensemble (n_estimators), which directly controls model capacity, and the maximum depth of each tree (max_depth), which regulates model complexity and prevents overfitting. Candidate values for n_estimators were 100, 200, and 300, while max_depth was varied between unlimited growth (None) and restricted depths of 10 and 20. Model selection was performed using 5-fold cross-validation on the training split with macro-F1 as the scoring metric.

To further exploit the combined feature space, gradient boosting models were employed. An XGBoost classifier, known for high speed (XGBClassifier), was optimized using RandomizedSearchCV with 5-fold stratified cross-validation and macro-F1 scoring. The search space was designed to balance model expressiveness with generalization. Specifically, the number of boosting rounds (n_estimators) varied between small ensembles of 50 trees and larger ensembles of up to 500 trees. The tree depth (max_depth) was tuned between shallow (3) and relatively deep (10) architectures, allowing the model to capture different levels of interaction complexity. The learning rate was tuned from conservative (0.01) to more aggressive updates (0.2). To improve robustness against overfitting, both the row subsampling ratio (subsample) and the column subsampling ratio (colsample_bytree) were tuned between 0.6, 0.8, and 1.0, introducing randomness during training. Finally, the gamma parameter, which controls the minimum loss reduction required for a split, was included with candidate values of 0, 1, and 5 to test different levels of pruning aggressiveness.

In parallel, a LightGBM classifier (LGBMClassifier) was trained, exploiting its histogram-based splitting strategy, which is particularly efficient for large datasets. The hyperparameter search space was similarly motivated by the need to balance accuracy and efficiency. The number of boosting iterations (n_estimators) ranged from 100 to 1000, reflecting both lightweight and high-capacity models. The learning rate was varied between 0.01 and 0.1 to test the trade-off between convergence speed and stability. The number of leaves per tree (num_leaves), a critical determinant of model complexity, was tuned between 15, 31, and 63, while the maximum depth of trees (max_depth) was constrained between 3 and 7 to avoid overfitting. Furthermore, the minimum number of samples required in a child node (min_child_samples) was set to 5, 10, or 20, ensuring that splits occurred only when sufficient data was present. Similar to XGBoost, subsampling ratios for rows and columns (subsample and colsample_bytree) were varied between 0.6, 0.8, and 1.0 to add regularization. Finally, L1 and L2 regularization terms (reg_alpha and reg_lambda) were included with values of 0, 0.1, and 1 to explicitly penalize overly complex models.

Finally, the same modeling pipelines were applied separately to the attribute-only features and to the embeddings, enabling a direct comparison with the fused feature representation. The corresponding findings are presented in the evaluation chapter.

6 Evaluation

Our validation setup followed the widely adopted „Hold-Out Validation“ setup with a train/validation/test split with early stopping, which is a standard model selection procedure. Nested cross-validation was not feasible under our strict inductive regime, since any mixing of the sets would have violated the leakage prevention constraints. We therefore committed to this stricter setup, even though it meant deviating from pipelines more familiar to us.

The results for the five model families, Logistic Regression, Multilayer Perceptron (MLP), Random Forest, XGBoost, and LightGBM, are summarized in table 3 below and include scores for the most performant models trained on only features, only embeddings, and the concatenation of both representations. In addition, Figure 3 visualizes the score distributions for the XGBoost, LightGBM, and Random Forest models, comparing their performance across three distinct input types. The evaluation was conducted using the macro-F1 and weighted average F1 scores because, although random undersampling was applied during training for Random Forest, the test set remained highly imbalanced for all models. Other standard metrics, such as accuracy or micro-F1, would fail to properly reflect the performance of the models on all, especially minority, classes since their scores would be dominated by the many samples of the majority classes. The macro-F1 score, however, treats all classes equally by averaging their individual F1 scores, giving the minor ones just as much weight as the major ones. This is crucial in our context of categorizing rare occurring fraudulent addresses. The weighted average F1 score was added to provide a more realistic picture of the overall predictive performance of the models, given the premise of class imbalance. The consistently higher weighted F1 score compared to the macro-F1 indicates a substantially better performance of the models on the dominant label groups, while the weaker results for minority labels are overshadowed once their low incidence is taken into account.

Table 3: Values of macro-F1 / Weighted Average F1

Structure / Method	XGBoost	LightGBM	Random Forest
only features	71 / 86	67 / 83	78 / 89
only embeddings	73 / 88	76 / 90	75 / 90
features + embeddings	86 / 95	85 / 95	80 / 92

The results clearly demonstrate that the fused feature representation outperforms the isolated use of features or embeddings across all model families. While Logistic Regression and MLP remained substantially weaker (≈0.35 and 0.60 macro-F1) and never exceeded a trivial majority-class baseline, the tree-based models showed a more differentiated picture. With features only, Random Forest reached the best performance with about 0.80 macro-F1, significantly outperforming both boosting methods. However, once concatenated representations were used, this pattern reversed: XGBoost and LightGBM achieved macro-F1 scores of 0.86 and 0.85, respectively, clearly surpassing Random Forest with 0.82. A paired t-test over five seeds confirmed that these improvements from concatenated over feature-only input were statistically significant for the boosting methods ($p < 0.001$), establishing that embeddings provide a complementary signal beyond handcrafted attributes.

Variance across seeds remained consistently low (±0.005–0.007), particularly for XGBoost and LightGBM in the concatenated setup, underlining the robustness of the results. At the same time, a per-class analysis revealed that the majority of classes achieved F1-scores above 0.90, while minority fraud classes frequently dropped below 0.40, explaining why weighted F1 was always higher than macro-F1. Finally, runtime analysis showed that training Logistic Regression and MLP was fastest but produced the weakest models, while Random Forest and boosting methods were moderately slower but remained efficient. The main computational bottleneck was the creation of embeddings, which required significantly more resources than the training of any classical model.

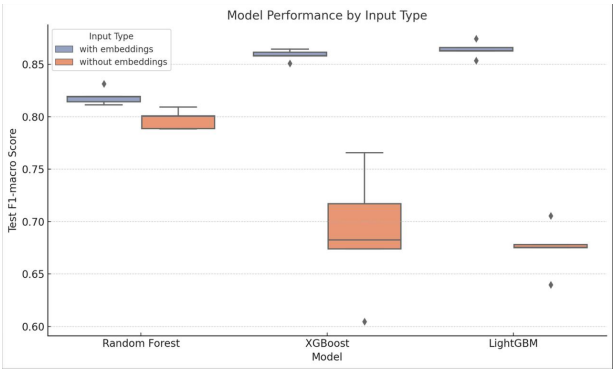

Figure 3: Model performance over 5 different seeds

7 Deployment

To deploy the trained models and to achieve our business goals, an interactive web application was developed. The main function of the application is to analyze one or more Bitcoin addresses. For this, it suffices that simply the name of the address or addresses is provided as a CSV file or entered as text. The system then displays the model's classification, the 1-hop transaction graph, and a selection of important information for investigations for every examined address. In addition to these fundamental features, an analytics dashboard was put in place to provide usage data and the distribution of anticipated categories. Moreover, the application includes a case management system to facilitate Know Your Customer (KYC) protocols and long-term monitoring. This makes it possible to combine several addresses into a single case for continuous monitoring and annotation. Users are also informed by an alert system whenever suspicious activity occurs, such as interactions with other high-risk addresses or abnormally large transactions. To facilitate quicker future lookups, a database containing the history of all examined addresses is being kept up to date.

To guarantee scalability and maintainability, the application was constructed using a microservice architecture. The backend was developed using Python with Flask as the framework, Gunicorn as a Python server for production and deployment, Docker for containerization, and a virtual environment for dependency injection. For data persistence, a production-ready database with PostgreSQL and object relational mapping (ORM) using Flask-SQLAlchemy with automatic table creation and migrations was implemented. The frontend user interface was created with JavaScript and the Svelte framework, as well as Chart.js for the visualizations. The machine learning services for data processing and inference were built with numerous libraries such as TensorFlow, Scikit-learn, Pandas, and NumPy. We implemented a hybrid approach for the classification, using separate models with and without structural features. The necessary features for classification were calculated using the formulas proposed in the documentation of BABD-13. Finally, the transaction data required for feature computation was sourced via the BlockCypher API.

8 Discussion

The results of our training, as displayed in the Evaluation, clearly showed an advantage of the combination of the features and the embeddings compared to the isolated matrices. This is technically reasonable, as embeddings capture structural information from the graph while the attribute-based features provide complementary descriptive information. By combining both sources, the models can exploit a richer and more expressive representation of the data.

Working with our dataset came with several substantial challenges, which we systematically and persistently addressed. One of the most unexpected and critical issues turned out to be data leakage during embedding creation. Preventing leakage became one of our central priorities as it was warping our results, and we implemented extreme measures to ensure a strict separation between training, validation, and test sets. This included enforcing an inductive split of the graph, such that no edges connected training nodes to validation or test nodes. While this setup gave us strong confidence in the reliability of our results, we must acknowledge that in complex graph structures, the possibility of subtle leakage can never be fully eliminated.

A second major difficulty was the severe class imbalance in the data. We tested several countermeasures, including undersampling of the training set and the use of class weights. Nevertheless, the imbalance heavily impacted performance, especially for simpler models and on smaller classes. Logistic regression, for instance, only achieved around 35% accuracy, while a multilayer perceptron (MLP) reached about 60% both of which were worse than a trivial majority-class baseline. These underwhelming results ultimately confirmed that more sophisticated approaches were necessary.

It is also important to acknowledge a limitation considering the feature set consistency. Due to the misleading information about the used Bitcoin blockrange for the BABD-13 dataset as mentioned in the data understanding and preparation, the few graphical features we extracted and used for modeling (S5: average shortest path length between nodes; S6: maximum diameter of the graph; S1-1: average in-degree) are based on the bigger transaction graph with the blockrange specified in their paper. The embeddings, which were concatenated with these features, however, were derived from a much smaller transaction graph, creating a mismatch of information. It is yet to be investigated whether this heterogeneity constituted a beneficial or detrimental effect for the predictive power of our model.

Another important limitation: our current setup does not yet exploit the full potential of the heterogeneous graph. Specifically, transaction nodes are not modeled as first-class entities but are instead treated only as edges between address nodes. This design choice simplified the setup but moderately left structural information unused. Extending the method to fully incorporate heterogeneous node types is therefore a clear next step for future work.

Despite the aforementioned methodological limitations, the project's primary business objective was successfully achieved. The use of fast APIs to get basic information about the analyzed address node and calculate the features and predictions in constant time makes the forensic analysis extremely fast and only requires the wallet's address. As shown, classifications achieve a high effectiveness with a weighted average F1-score of 95% and a macro F1-score of 86%. Through the microservice architecture and batch analysis extension, the application provides the scalability needed. The latter, together with the implementation of an alert and address management system, saves on manual labour concerning address analysis and regular checkups on the addresses. Therefore, the application succeeds in meeting our criteria of success.

We were also able to provide a beneficial answer to the question raised by Weber et al. (2019), whether it is possible to combine a Random Forest with a graphical neural network (p. 6), given that our concatenated feature matrix provides considerably better results than any of the isolated matrices on their own. In achieving this, we provide a strong argument against their statement that a "simple" concatenation of the node features with the embeddings would only help "marginally" (Weber et al., 2019, p. 6).

In summary, despite major obstacles such as data leakage and class imbalance, we succeeded in constructing a reliable and transparent pipeline. Our insistence on strict separation and robust validation came at the cost of higher complexity and, at times, lower short-term performance, but ultimately ensured trustworthy results. The evaluation thus demonstrates both the effectiveness of our approach under difficult conditions and the clear directions in which it can be further optimized.

9 Conclusion

Our final approach provides the accuracy, speed, automation, and scalability needed to enable law enforcement,

regulated cryptocurrency exchanges, and other actors who have to comply with regulatory requirements (European Commission, 2024a; 2024b), to perform financial forensics on the Bitcoin blockchain to find and condemn their criminal application, such as money laundering.

Future feature extensions could include an integration into Anti-Money Laundering systems, automatically generating compliance reports for financial authorities, and integrations of wallet blacklists from law enforcement. It would also be possible to provide this service to other developers as a public API and to apply this technology to different chains, also allowing for the possibility of cross-chain analysis, which also includes money flows that switch between different cryptocurrencies.

A Calculations for Feature Selection

A.1 Chi-squared test

This statistical hypothesis test describes the difference between the observed and expected data. It examines whether two categorical variables, in our case feature and label, are statistically independent. We calculate the chi-squared score using the observed value x_i and the expected value m_i.

$$\chi^2 = \sum_{i=1}^{k} \frac{(x_i - m_i)^2}{m_i}$$

If χ^2 is greater than the critical value, the null hypothesis gets rejected i.e. the statistically significant dependency between the feature and the label exists. We calculate the critical value using the significance level α and the degree of freedom df which is fairly high in our case. For our final selection of 8 features, each fulfills this requirement:

$$\chi^2 > \chi_{\alpha,df}$$

Therefore all 8 features influence the label significantly. Figure 4 shows the chi-squared scores for our 8 best features. It shows that the highest statistical dependence between the label and the features, exist between the label and CI2a32-2, PAIa13 and S5.

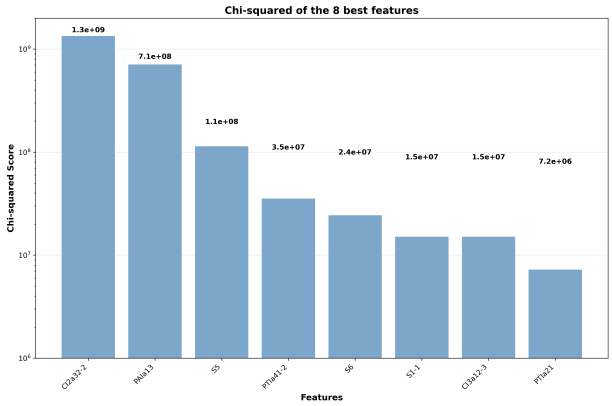

Figure 4: Chi-squared test for the 8 best features

A.2 F-test

The F-test is a statistical test that compares variances and measures how strong features differ between classes. We calculate the F-statistic by dividing the variances of the features we want to analyze. Note, that the higher variance feature is the numerator, resulting in F being greater than or equal to 1.

$$F = \frac{S_2^2}{S_1^2} = \frac{\frac{1}{n_2-1}\sum_{i=1}^{n_2}(X_{2,i} - \bar{X}_2)^2}{\frac{1}{n_1-1}\sum_{i=1}^{n_1}(X_{1,i} - \bar{X}_1)^2}$$

After calculation, F gets compared with the critical F-value from the F-distribution that depends on the significance level α and the degrees of freedom of both the numerator and denominator. Our final 8 features have all rejected the null hypothesis and have therefore shown that they have an excellent discriminative power, since F is greater than the critical F-value for all of them. Figure 5 shows the F-test scores for these features and reveals that S5 is the best performing feature regarding the F-test by a wide margin.

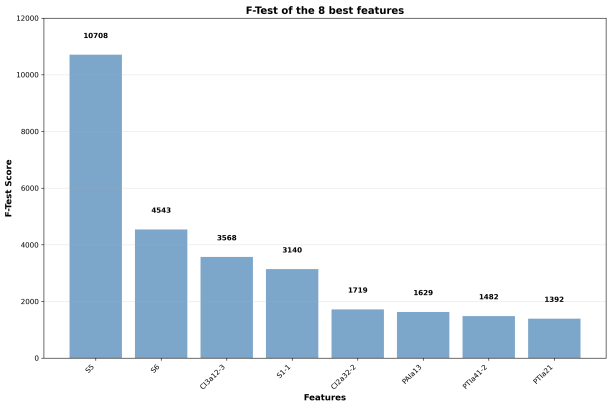

Figure 5: F-test for the 8 best features

A.3 Pearson Correlation Coefficient

The Pearson correlation coefficient is an indicator that measures linear correlation between two sets of data, in our case between feature and label. Results have a value between -1 and 1 and therefore are a normalized measurement of strength and direction of the covariance. We calculate the coefficient by dividing the covariance of the to be examined random variables X, Y divided by the product of the standard deviations:

$$\rho_{X,Y} = \frac{\mathrm{cov}(X,Y)}{\sigma_X \sigma_Y} \tag{1}$$

$$= \frac{\mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y]}{\sqrt{\mathbb{E}[X^2] - (\mathbb{E}[X])^2}\sqrt{\mathbb{E}[Y^2] - (\mathbb{E}[Y])^2}} \tag{2}$$

Figure 6 shows the result of the Pearson correlation coefficient calculated for our 8 best features. We can see that all features have a positive correlation to the label, with the structural features S6, S5 and S1-1 showing the highest correlation.

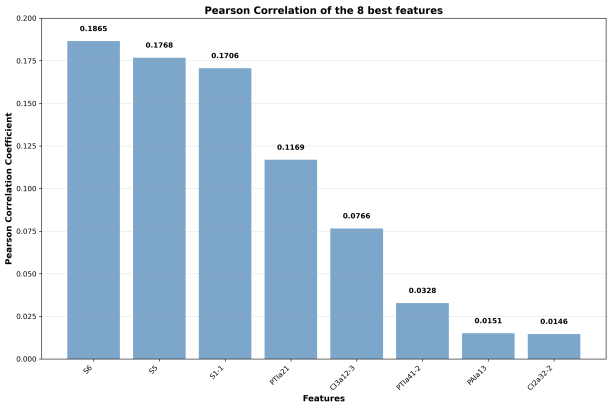

Figure 6: Pearson Correlation for the 8 best features

A.4 Mutual Information

Mutual information describes the dependence between two random variables and can be applied in our case to measure how much information a feature contains about the label. One of the advantages of mutual information is that it also takes non-linear information into account. The higher the non-negative mutual information, the more information can be deducted from one variable by the other. Since we use discrete variables, we use the following formula to calculate the mutual information:

$$I(X;Y) = \sum_{y \in \mathcal{Y}} \sum_{x \in \mathcal{X}} P_{(X,Y)}(x,y) \log \left(\frac{P_{(X,Y)}(x,y)}{P_X(x) P_Y(y)} \right)$$

Figure 7 shows the mutual information for our 8 best features. Again, the structural features S5, S6 and S1-1 show the highest importance, by containing the most information about the label.

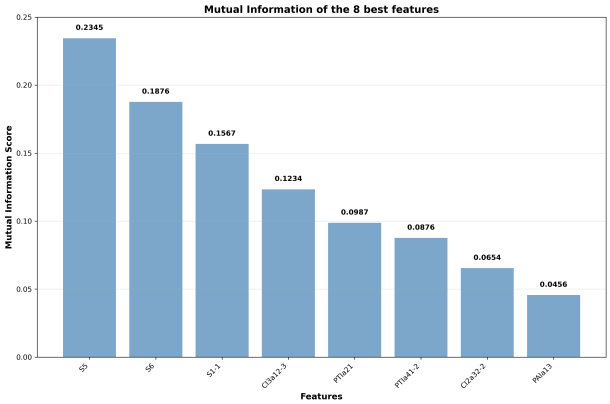

Figure 7: Mutual information for the 8 best features

References

Andersson, D. and Olsson, A. (2024). *The Dark Flows of Cryptocurrency – an overview of money flow behaviors in Bitcoin transactions related to online criminal activities and Bitcoin mixers*. [Online]. Available at: https://www.diva-portal.org/smash/get/diva2:1873090/FULLTEXT01.pdf

Asiri, A. and Somasundaram, K. (2025). Graph convolution network for fraud detection in bitcoin transactions. *Scientific Reports*, 15, p. 11076.

Çağlayan, M. and Bahtiyar, Ş. (2022). Money laundering detection with Node2Vec. *Gazi University Journal of Science*, 35(3), pp. 854–873.

Conlon, T. and McGee, R. J. (2020). Betting on Bitcoin: Does gambling volume on the blockchain explain Bitcoin price changes? *Economics Letters*, 191, p. 108727.

Elliptic (2019). *Elliptic Data Set*. Kaggle. [Online]. Available at: https://www.kaggle.com/datasets/ellipticco/elliptic-data-set

Elmougy, Y. and Liu, L. (2023). Demystifying Fraudulent Transactions and Illicit Nodes in the Bitcoin Network for Financial Forensics. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pp. 3979–3990.

European Commission (2024a). *Anti-money laundering and countering financing of terrorism*. [Online]. Available at: https://finance.ec.europa.eu/financial-crime/anti-money-laundering-and-countering-financing-terrorism-eu-level_en

European Commission (2024b). *Crypto-assets*. [Online]. Available at: https://finance.ec.europa.eu/digital-finance/crypto-assets_en#legislation

Hu, Y. et al. (2019). Characterizing and detecting money laundering activities on the bitcoin network. *arXiv preprint arXiv:1912.12060*.

LEMONX (2022). *BABD-13*. Kaggle. [Online]. Available at: https://www.kaggle.com/datasets/lemonx/babd13

Liu, J. et al. (2023). Graph embedding-based money laundering detection for Ethereum. *Electronics*, 12(14), p. 3180.

Nakamoto, S. (2008). *Bitcoin: A Peer-to-Peer Electronic Cash System*. [Online]. Available at: https://bitcoin.org/bitcoin.pdf

Scharnowski, S. (2024). Dark web traffic, privacy coins, and cryptocurrency trading activity. *Finance Research Letters*, 67, p. 105875.

Weber, M., Domeniconi, G., Chen, J., et al. (2019). Anti-money laundering in bitcoin: Experimenting with graph convolutional networks for financial forensics. *arXiv preprint arXiv:1908.02591*.

Xiang, Y. et al. (2024). BABD: A Bitcoin Address Behavior Dataset for Pattern Analysis. *IEEE Transactions on Information Forensics and Security*, 19, pp. 2171–2185.

Y-Xiang-hub (2022). *Bitcoin-Address-Behavior-Analysis*. GitHub. [Online]. Available at: https://github.com/Y-Xiang-hub/Bitcoin-Address-Behavior-Analysis

Yuan, Q. et al. (2020). Detecting phishing scams on ethereum based on transaction records. In *2020 IEEE international symposium on circuits and systems (ISCAS)*.